



## **S25 - EA1. DIAGRAMA DE CLASES**

Tecnología en desarrollo de software.

### **ASIGNATURA:**

Programación II: Orientada a objetos avanzada.

### **DOCENTE:**

Boris Alberto Salleg Royero

### **INTEGRANTES:**

Valentina Stephany Kassar Acuña

Maria Ines Julio Navarro

Lina marcela Muñoz

(GRUPO 5)

**INSTITUCIÓN UNIVERSITARIA DIGITAL DE ANTIOQUIA**

**2026**

## ÍNDICE

### 1. Introducción.

1.1 ¿Qué es un diagrama de clases UML y su importancia en el diseño de software orientado a objetos?.

1.2 Breve mención del sistema a modelar (Biblioteca: libros, autores, préstamos,etc).

1.3 Propósito del trabajo colaborativo y las herramientas empleadas.

### 2. Objetivos.

2.1 Objetivo general y objetivos específicos alineados con la actividad (abstracción, encapsulamiento, herencia, polimorfismo, principios SOLID).

### 3. Desarrollo.

3.1 Descripción de las clases identificadas (Libro, Autor, Préstamo, LibroDígital, etc.)

3.2 Justificación de las relaciones utilizadas (asociación, composición, herencia, agregación).

3.3 Explicación de aplicación de los principios de cohesión, bajo acoplamiento y SOLID.

3.4 Captura de pantalla del diagrama.

### 4. Conclusiones.

4.1 ¿Qué se aprendió?.

4.2 ¿En qué aporta a su desempeño profesional?.

### 5. Enlaces verificados.

5.1 Repositorio github

5.2 Enlace video explicativo.

## INTRODUCCIÓN

Nuestro grupo desarrolló este trabajo con el propósito de aplicar los principios de la programación orientada a objetos mediante el diseño de un diagrama UML.

### **1.1 Diagrama de clases UML y su importancia en el diseño de software orientado a objetos.**

El lenguaje unificado de modelado (UML), y en particular el diagrama de clases, constituye una herramienta fundamental en el diseño de software orientado a objetos. Su importancia es muy grande ya que radica en la forma en que permite representar de forma visual, estructurada y estandarizada, facilitando así la abstracción de los componentes necesarios, el encapsulamiento de datos y teniendo la definición clara de las responsabilidades entre todo los objetivos antes de realizar la codificación.

### **1.2 Breve mención del sistema a modelar.**

En el presente trabajo se aborda el diseño y modelado UML para así poder llevar a cabo una solución orientada a objetos de un sistema de gestión de biblioteca. Se logra ver que el dominio contempla la abstracción de entidades esenciales como Libro, Autor, Préstamo, LibroDigital y más entidades que pudimos generar a través del sistema de gestión, llevando así a modelar sus atributos y métodos.

### **1.3 Propósito del trabajo colaborativo y las herramientas empleadas.**

Este trabajo tiene como propósito llegar a fortalecer las competencias de diseño orientado a objetos, llevando a promover el trabajo en equipo de una forma distribuida pero llegando así a un mismo nivel de aprendizaje sobre lo que se está desarrollando.

Para desarrollar el modelo se empleó una herramienta de diagramación (PlantUML), permitiéndonos así la construcción conjunta de la solución, llevando a utilizar también una plataforma donde pudimos grabar para demostrar nuestro dominio con el tema.

Esto nos llevó a integrar buenas prácticas aplicables al entorno profesional de desarrollo de software.

## **OBJETIVOS**

### **2.1 Objetivo General.**

Diseñar y documentar un sistema de biblioteca mediante un diagrama de clases UML aplicando principios de programación orientada a objetos y buenas prácticas de diseño.

### **2.2 Objetivo Específico.**

- Aplicar abstracción en la definición de la clase base Persona
- Implementar encapsulamiento en los atributos de las clases
- Utilizar herencia en las relaciones entre Persona, Autor y Usuario, así como entre Libro y LibroDigital.
- Demostrar polimorfismo mediante la interfaz CatalogoItem.
- Incorporar principios SOLID para garantizar cohesión y bajo acoplamiento en el diseño.

## DESARROLLO

### 3.1 Descripción de las clases identificadas (Libro, Autor, Préstamo, Libro Digital, etc.)

En esta clase los factores principales son:

- Libro su función es ser leído y prestado.
- Autor del libro y cuántos libros tiene en la biblioteca.
- Préstamo solicitud por parte del usuario.
- Biblioteca institución física o digital para ordenar libros.
- Estado préstamo el encargo de decir si está activo, devuelto o vencido.
- Libro digital un archivo de texto
- Persona que va a estar a cargo de que el libro salga de la biblioteca.

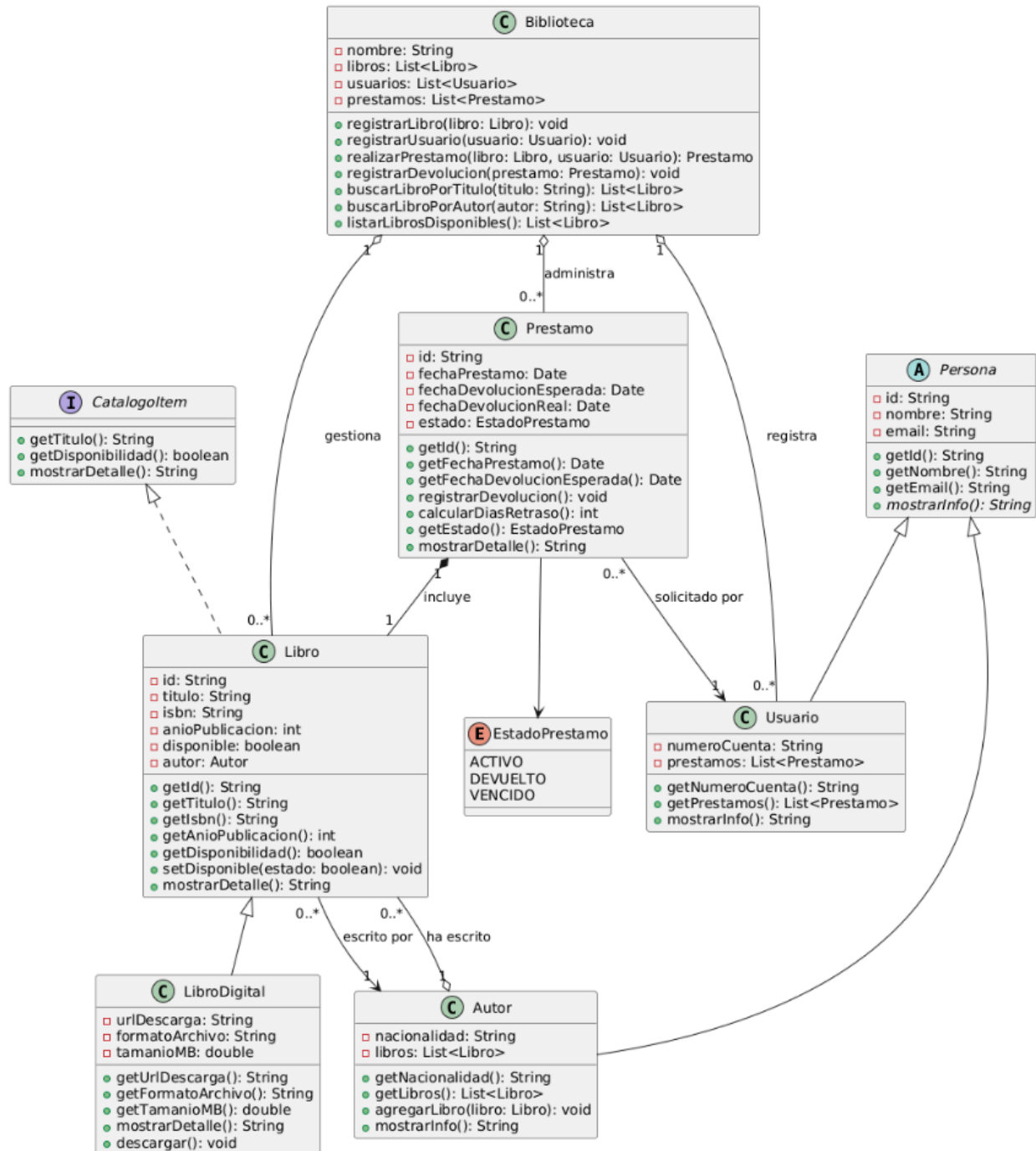
### 3.2 Justificación de las relaciones utilizadas (asociación, composición, herencia, agregación).

- La **asociación** es una de las relaciones más comunes en los diagramas de clases. Se presenta cuando una clase contiene una referencia a una o más instancias de otra clase, lo que indica que existe algún tipo de vínculo entre ellas.
- La **composición** nos ayuda a entender cómo funciona y cuando ocurren las acciones.
- **Herencia** utilizar clases ya realizadas para obtener otros atributos nuevos.
- La **agregación** es un tipo de relación entre clases en la programación orientada a objetos donde un objeto contiene a otro, pero ambos tienen vidas independientes.

### 3.3 Explicación de aplicación de los principios de cohesión, bajo acoplamiento y SOLID.

- **Cohesión:** cada clase tiene una responsabilidad clara (ej. Biblioteca gestiona, Préstamo controla fechas).
- **Bajo acoplamiento:** las clases interactúan mediante métodos públicos, evitando dependencias rígidas.
- **SOLID:**
  - **Single Responsibility:** cada clase cumple una función específica.
  - **Open/Closed:** el sistema puede extenderse (ej. nuevos tipos de libros) sin modificar las clases existentes.
  - **Liskov Substitution:** LibroDigital puede sustituir a Libro sin romper el sistema.
  - **Interface Segregation:** CatalogoItem define un contrato claro para ítems del catálogo.
  - **Dependency Inversion:** las clases dependen de abstracciones, no de implementaciones concretas.

### 3.4 Captura de pantalla del diagrama.



## CONCLUSIONES

### 4.1 ¿Qué se aprendió?

Como equipo, logramos comprender la importancia de los diagramas UML para visualizar sistemas antes de programarlos y se logró representar un sistema mediante diagramas UML aplicando de manera correcta los principios de la POO y las buenas prácticas; se comprende cómo las relaciones entre las clases permiten reflejar con claridad el funcionamiento del sistema en la realidad. Así mismo, se logró comprender la importancia práctica de los principios SOLID. Observando como la abstracción y la aplicación de interfaces permiten estructurar un sistema extensible y fácil de mantener.

### 4.2 ¿En qué aporta a su desempeño profesional?

Fortaleció la capacidad de modelar sistemas complejos y sentó las bases para abordar el desarrollo de software desde una fase previa de análisis y arquitectura, reduciendo errores antes de la codificación. De igual manera, la experiencia con herramientas modernas de diagramación como plantUML y la dinámica de trabajo colaborativo distribuidos permitieron ejercitar habilidades claves a cada una para el ámbito laboral actual.

## ENLACES VERIFICADOS

### 5.1 Repositorio github

[https://github.com/DesarrolloMarce25/EA1\\_DiagramaClases\\_Grupo5.git](https://github.com/DesarrolloMarce25/EA1_DiagramaClases_Grupo5.git)

### 5.2 Link video

[https://drive.google.com/file/d/1dZB2956C\\_jXDU0pzLRVC7BvqmR0u-Oep/view?usp=sharing](https://drive.google.com/file/d/1dZB2956C_jXDU0pzLRVC7BvqmR0u-Oep/view?usp=sharing)